

MODULE 1 · WEEK 2

Tree Models & Neural Networks

The ML modeling framework refreshed on Boston Housing, then tree-based models and a first look at neural networks.

Lectures 16–17 · 64–68

This week

- 1 **The modeling framework** — regression & classification refreshers
- 2 **Tree-based models** — Decision Tree, Random Forest, Gradient Boosting
- 3 **Neural networks** — what they are and how they learn
- 4 **Gradient descent & backprop** — the engine under the hood
- 5 **MLPRegressor & MLPClassifier** — neural nets the sklearn way

The modeling framework (refresher)

- Same five steps every time: **read** → **split** → **scale** → **fit** → **evaluate**.
- Regression predicts a number (house price); classification predicts a category (above/below median).
- We build the same mental model twice on Boston Housing so the pattern becomes muscle memory.

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42)
```



Evaluation: regression vs. classification

- **Regression:** MAE (avg error in real units), MSE, R^2 (variance explained).
- **Classification:** confusion matrix → precision, recall, F1.
- Always compare train vs. test — similar scores mean the model generalizes.

Tree-based models

- **Decision Tree** — a flowchart of if/then splits; fast and interpretable, but overfits.
- **Random Forest** — hundreds of trees on random subsets, averaged; robust and hard to break.
- **Gradient Boosting** — trees built sequentially, each fixing the last one's errors; often top accuracy.
- These are the most widely used models in applied data science.

```
from sklearn.ensemble import RandomForestRegressor
rf = RandomForestRegressor(n_estimators=300, random_state=42)
rf.fit(X_train, y_train)
```

Key idea Trees → forests → boosting: each step trades a little simplicity for a lot of accuracy.



From trees to neural networks

- A neural network is layers of simple units (neurons), each a weighted sum passed through an activation.
- Stacking layers lets the network learn **highly nonlinear** relationships automatically.
- Same job as before — regression or classification — just a more flexible function.

How neural nets learn: gradient descent & backprop

- **Loss** measures how wrong the prediction is. Training = making the loss small.
- **Gradient descent** nudges every weight in the direction that lowers the loss, step by step.
- **Backpropagation** is the bookkeeping that computes each weight's share of the error efficiently.

Key idea Backprop assigns blame; gradient descent acts on it — repeat until the loss stops dropping.

Neural nets in sklearn — MLPRegressor

- **MLP** = Multi-Layer Perceptron, the classic feed-forward neural network.
- Scale your inputs first — neural nets are very sensitive to feature ranges.
- Set hidden layers, an activation, and let it train.

```
from sklearn.neural_network import MLPRegressor
nn = MLPRegressor(hidden_layer_sizes=(64, 32),
                  max_iter=500, random_state=42)
nn.fit(X_train_scaled, y_train)
```


MLPClassifier & best practices

- **MLPClassifier** is the same idea for categories — outputs class probabilities.
- Best practices: **scale inputs**, watch for non-convergence (raise `max_iter`), and start small.
- Don't reach for a neural net when a Random Forest already wins — simpler is better.

```
from sklearn.neural_network import MLPClassifier
clf = MLPClassifier(hidden_layer_sizes=(100,),
                    max_iter=500, random_state=42)
```



Lab this week

- Load Boston Housing, split/scale in clean functions, and fit Decision Tree, Random Forest, and Gradient Boosting — for both regression and classification.
- Evaluate with MAE/ R^2 and the classification report; compare the models fairly.

In the labs repo: `Module1/Week2_TreeModels/boston_regression.py` · `Module1/Week2_TreeModels/boston_classification.py`



Key takeaways

- Every model follows read → split → scale → fit → evaluate.
- Trees → Random Forest → Gradient Boosting: increasing power, decreasing interpretability.
- Neural nets learn via gradient descent + backprop by repeatedly shrinking the loss.
- Use MLPRegressor / MLPClassifier in sklearn — always scale inputs first.
- Reach for the simplest model that does the job.